# Measured figures behind the code

Every number in `tsp/` that came from a measurement rather than a specification, with the conditions it was taken under. Comments in `tsp/` cite the headings here instead of carrying the derivations, so a constant can be re-derived without reading the module it lives in.

Firmware behaviour the manuals contradict or omit is in `UNDOCUMENTED.md`. This file is the app's own timings, geometry and arithmetic.

Instrument: DMM6500, firmware 1.7.17a, unless stated.

## Reading a sample out of a reading buffer

`buf.readings` is a firmware proxy, so every index is an IPC round trip. Three independent measurements agree:

| figure | how it was taken |
|---|---|
| **21.73 us** | the read component of a 20 000-sample decode step, timed per phase |
| 22.75 us | derived from the 0.455 s a capture that `acq_copy_out` spends |
| 22.82 us | timed directly, 20 000 reads in one loop |

**Independent of buffer style**, so the cost is the round trip and not the payload: `STYLE_FULL` stores more per reading and reads within 0.07 % of `STYLE_STANDARD` (22.836 us against 22.820, three interleaved runs each).

**Acquisition is the opposite** -- 20 000 samples take 207.96 ms into a `STYLE_STANDARD` buffer and 221.65 ms into `STYLE_FULL`, +6.58 % at 8.3 sd. So payload size costs time where it is written and is invisible where it is read. `buffer.make` (1.8 ms) and `buf.clear()` (0.17 ms) are style-independent.

### Streaming by real-time extraction is not possible

9600 baud at 10 samples/bit needs 96 000 samples a second: ~10 us a read with nothing left for decoding, ~5 us to leave half the budget. 21.73 us is 2.2x and 4.3x over those. Double buffering cannot help, the deficit being throughput rather than a gap between captures, so the app decodes one window at a time.

## Decode phase timings

Timed per phase with `tools/bench_priming.py`, 9600 baud, W = 20 000:

| phase | time | what it is |
|---|---|---|
| levels | 0.958 s | the decimated pass, `ck_level_n` samples |
| reading | 0.530 s | the contiguous lead-in read |
| edges | 0.101 s | `sig_edges` + `sig_idle` |
| format | 1.043 s | the format search, the most expensive per sample |
| window | 0.685 s | one decode slice, x20 |

Every phase is proportional to W, so one per-sample cost bounds all five, which is what makes the latency bound real. Timing only `sig_edges` and `ua_run` over a window already in a Lua table understates the total threefold, because the buffer read is part of the cost.

### ck_smp_us = 73

The format phase scales with FRAMES, and samples-per-bit varies 2:1. At W = 8653: 0.451 s at 9600 baud (8.33 sa/bit) against 0.634 s at 250 kBd (4.0 sa/bit). Two points give 32.5 us/sample + 1.63 ms/frame, so 52 us/sample at 8.33 sa/bit and 73 at 4.0. The worst is used: one always-safe constant beats two that must be evaluated before the format is known. Costs ~30 % window width at low rates -- throughput, not correctness.

### ck_win_us = 48

One 20 000-sample decode step is 0.685 s, i.e. 34.25 us a sample, of which 21.73 is the buffer read and does not scale with frames. Only that one operating point was timed, so the worst case comes two ways: scaling the whole figure as the format phase scales from 8.33 to 4 sa/bit (x 73/52) gives 48, and holding the read fixed while doubling the frame-proportional remainder gives 47. The higher is used.

### ck_slice_n = 1

Samples per press = W x n, and n = floor(budget / (W x ck_win_us)), so raising n by narrowing W cannot buy more than budget / ck_win_us samples per press: 9375 at 0.45 s against 6164 today. That 1.52x is the whole prize and taking it needs W <= 4687, i.e. n = 2. Three constraints then do not fit at 0.5 s -- n = 2, a W that stays a useful width (>= 5000, asserted in `test_serial.lua`), and the 0.9 margin. Any two hold: n = 2 with W = 5000 spends 0.480 s of the 0.5 s bound, inside it but on 4 % margin rather than 10 %.

## Window geometry

### Overlap

`ua_run` stops when a frame runs off the end of its array, so a window ending at W leaves at most one partial frame, whose start t0 satisfies t0 > W - framebits*T. The next window must therefore begin at or before W - framebits*T, giving `overlap >= framebits * T` as the floor.

Two further frames plus `maxback` buy two things. `sig_cross` walks back up to `maxback` samples to find a crossing and that walk is truncated at a window's leading edge, so the first edge or two can be mispositioned; with margin they sit in already-emitted territory and `tmin` discards them. The seam also falls mid-overlap rather than at its rim, so de-duplication never decides a near-tie.

At 8.68 samples/bit and 10-bit frames: ceil(260.4) + 64 = **325 samples, 1.6 %** of a 20 000-sample window.

### Constant stride

The window loop advances by a constant stride rather than resuming where the last frame ended. That costs ~1.6 % in redundant decoding -- the overlap, once per window -- and buys a window count known before the run starts, which is what makes the progress indicator honest.

## Level measurement

### Why ck_level_n is 20 000

How many samples the level measurement decimates the recording down to:

| count | bins | modal | threshold | sig_levels |
|---|---|---|---|---|
| 982 | 61 | 0.585 | 3.263 V | accept -- threshold ABOVE the high level |
| 1963 | 122 | 0.325 | 1.651 V | accept |
| 3925 | 245 | 0.176 | 1.647 V | accept |
| 5233 | 327 | 0.235 | 3.295 V | accept -- wrong again |
| 7850 | 490 | 0.137 | 1.648 V | refuse |
| 10466 | 512 | 0.146 | 1.645 V | refuse |
| 31397 | 512 | 0.145 | 1.645 V | refuse |

`sig_levels`' bin count is floor(n/binocc) capped at nbins, so it saturates at n = 8192 and every count past that measures the same thing. Below saturation the coarse bins sometimes pick the wrong pair of humps outright, so a small count is not merely noisier. `sdec.n` is past saturation and is frame mode's own population, so chunked mode accepts and refuses a line where frame mode does.

### Why the contiguous fallback is spaced across the recording

Decimation can alias past a sparse burst: a short message in a long recording -- the traffic the streaming modes exist for -- reduces to a handful of samples once the step is 140, and can reduce to none. Reading only the first window covers **20 000 of the 3 413 334** a full 32 kB recording holds, 0.6 % of the burst positions, hence `ck_probes` contiguous probes spaced evenly. The decimated view is still tried first, so the drift property that motivated decimation is untouched.

## Priming and the pre-trigger reserve

### An armed record opens on silence

`LoopUntilEvent` keeps a pre-trigger reserve ahead of the trigger, and on a device that had not started talking when Capture was pressed, every reading of that reserve is the line the arm was waiting on: **71 865 readings of a 32 kB recording's 1 437 294**, which is 3.6 windows of 20 000. So the first window can be entirely the wait.

Measured at 9600 Bd locked: a device starting at +3.0 s gave **0 bytes** and "no frame format fits the capture", where the same recording of one starting at +0.3 s gave **21 933**.

### ck_prime_tries = floor(leadmax / W) + 2

Plus two, never fewer than two, and the 8 kB mode is why: its reserve is 2 035 readings short of a window, so its first window is 90 % the wait. The window straddling the end of the reserve holds a sliver of signal which need not frame, so the try after the reserve is the first certainly on the device's own traffic. **5 tries for the 32 kB mode, 2 for the 8 kB mode.**

It is a bound because every try costs a window. Over the reserve a try is a read, an edge pass and a format search with nothing to search -- 0.63 s at W = 20 000 -- and on a window full of frames that do not fit it is the whole 1.67 s. The worst case is 5 tries at 32 kB: **8.4 s to refuse a recording that takes 1.67 s to refuse on one try.**

### Why sig_idle must exclude the leading run on an armed record

The priming window of an armed recording is the reserve, so its leading run is the wire before the device started, not the line's idle level. Left unset, `sig_idle` takes that run as the longest in the window and the polarity prior comes out of it -- and because priming's own decode then frames at that polarity, the format it returns is inverted and the whole recording inherits it.

Measured, an 8 kB recording armed on silence with the generator switched on at +3 s: leadrun 17 968 against a 17 965-sample reserve, run0 17 968, run1 232, idle 0, invert true, **2110 of 5484 bytes bad**. The byte count is right, which is why a count-only assertion passes on it.

## Mode caps

### ck_maxbytes = 32768

One number for every baud rate, so the UI states one expectation. 32 kB is 400-500 lines of 64-column debug text. 32 rather than 64 kB because the worst case then needs **3.41 M samples**, under half the reading pool, so the defbuffers at their 100 000 defaults leave room -- and where they do not, `sdec.bufmax_for()` shortens the window instead of refusing.

### Window reuse between priming and decode

`ck_prime_step`'s read phase fills `win[1..cnt]` from position 1, and nothing between it and `ck_decode` writes into the table, so re-reading the first window costs W proxy round trips for values already in hand -- **0.435 s at W = 20 000**. The reuse is conditional on four things, because a format search that found nothing in the reserve advances a window at a time and the table can hold samples from the middle of the record.

## The USB file API

### No file call raises

Measured with the key present. Every failure posts an event instead:

| call | returns | posts |
|---|---|---|
| `file.write` to a closed handle | returns | 2200 |
| `file.flush` on a closed handle | returns | 2211 |
| `file.close` on a closed handle | returns | 2202 |
| `file.read` on a closed handle | nil | 2201 |
| `file.read` on a WRITE handle | nil | 2201 |
| `file.open` on a bad path | nil | 2216 |
| `fs.is_file` bad path or garbage | false | nothing |
| `file.write(fh, {})` -- a table | returns | 1138, 'parameter 2, expected type string' |

The table row is the strongest: even a Lua argument-type error becomes a posted event. Across every shape measured -- dead handle, wrong mode, bad path, wrong argument type -- a `pcall` round a file call returned true, so a verdict taken from one is a constant. **Presence is the only file failure this application can observe.**

The rule is measured, not universal: these are argument shapes on a healthy key. A key that is present and REFUSING -- full, or write-protected -- has never been exercised. `vendor/repro-04b-full-key.tsp` would settle it.

### A key pulled from under a running app

Key out, handle still open:

| call | pcall | posts |
|---|---|---|
| `file.write(stale, s)` | ok = TRUE, err = nil | 2200, 'File write error' |
| `file.flush` / `file.close` | ok = true | nothing |
| `file.open(path, ...)` | returns nil | 2224, 'USB flash device not present' |
| `file.usbdriveexists()` | returns 0 | nothing |

`file.usbdriveexists()` costs **237 us** with a key in, names no path and posts nothing, so it is askable per line, per capture and per panel refresh.

### file.mkdir

Returns nothing, never raises, and posts **2208** on failure -- including for a name that already exists, so a speculative mkdir files one event per call. A bare name always fails; `file.mkdir('/usb1/SERDEC')` succeeds silently. The reference's own `file.mkdir("TestData5")` example is wrong. Opening a missing path posts **2205**, so probing for the directory is not free either.

The only silent way to tell is to read `/usb1`, which opens for read and yields its raw FAT directory table. With no key even `/usb1` posts 2205, hence `usbdriveexists()` first.

### Why the directory is called SERDEC

A name of eight characters or fewer in upper case is a single contiguous 8.3 entry, found by one `string.find`. FAT splits a long name across fields of 5, 6 and 2 characters with record metadata between them, so no search for the whole name matches:

```
SERIALFILES = false     no plain entry exists for an 11-character name
utf16_full  = false     the UTF-16LE long name is NOT contiguous
utf16_5     = true      only its first five characters are
SERIAL~1    = true      a tilde alias exists, but its number is not predictable
```

The root listing is **65536 bytes**, measured, which is why a confirmed directory is cached.

### fs.is_file against the open probe

`file.open(path, MODE_READ)` uses the failure as the answer and posts 2205 at severity ERROR, the one severity `showevents` still raises a modal box for -- so finding a free name would produce one on every Save. `fs.is_file()` answers silently: `eventlog.getcount()` stays 0 across an existing and a missing path.

## LIN

### The checksum's blind spot

Modulo-255 makes 0xFF congruent to 0x00, so swapping one for the other is invisible in every byte position: **0 of 256 detected across all 64 IDs and 4 positions.** With PID 0x11, data 0xFF and 0x00 both give checksum 0xEE. An agreeing checksum is therefore no proof that a byte the UART layer flagged arrived intact.

The checksum has **never been checked against a real frame** -- hand-computed cases only. `li_pid` is checked against the known headers: ID 0x3C -> PID 0x3C, 0x3D -> 0x7D, 0x00 -> 0x80, 0x01 -> 0xC1.

### Break detection

`li_minbreak = 11` bit times. A break plus delimiter is at least 12 and a byte butted against the next is 10, so 11 separates them. Corroboration rather than an independent test: `li_break` also requires value 0x00 and a failed stop bit. What survives all three is a 0x00 whose stop bit flipped inside a stream with 2+ bit times of space, which splits one frame in two and fails both checksums.

### Wiring

A 12 V single-wire bus against a 10 V best digitize range needs a 2:1 divider, 47 k + 47 k. Nothing below 20 k per leg, or a slave-only segment's pull-up reads recessive as dominant. A 24 V commercial-vehicle bus needs more than 2:1. `lin-divider-panel.png` is the drawing.

Distinguishing a divided bus from a transceiver TTL pin cannot work: 6 V against 5 V, and both decode anyway. So the panel note tests the recessive level only.